

The `toPromise()` method on `Observable` converts it to Javascript Promise object. As in `Observable` case, here also we are converting the `Http Response` object to `User[]` array. If any error occurs in `HTTP` call, `catch` function on `Observable` invokes the `handleError` method that returns the rejected Promise object and returns it to consumer of `getUsers()`.

Sending data to server using Http POST

`Http post` is used for sending data to server. In following example we are creating new user. The user's name is send to server in `JSON` format, then server replies with updated user object in `JSON`.

Code can be found here: <http://pastebin.com/Ymd52rVO>

We require `Header` and `RequestOptions` class from '@angular/http' and are imported in beginning. The `Http.post()` method takes web url as its first parameter like `http.get()` method.

The second parameter is a `User` object converted to `JSON` format as a string using Javascript's `JSON.stringify()` method. Angular `Http` client will convert this string into byte array using default string encoding and send it to server.

The server receives this byte array but won't understand how to interpret this data. So a `Content-Type` http header must be added to request for server to understand the data format. In Angular we use `Header` class from '@angular/http' to add http headers. First two lines of `createUser()` method creates the `Header` object for 'Content-Type' header for `JSON` format. The `post` method takes `RequestOptions` as its second optional parameter. `RequestOptions` allows further configuration of the http request object. Here we are using it's "header" field to add new `Content-Type` header to request. this options object is then passed as second paramter to the `http.post()` method.

The http response handling is same as `getUsers()` method. We choose Javascript Promise as result to be exposed to consumer of `createUser()` method. The server create API returns the updated `User` object in `JSON` format with few additional data. We parse return data as `JSON` and convert it to `User` object. Error handling is same as `getUsers()` service method.

Updating data on server using Http PUT

Like `http POST`, `http PUT` is also used to send and receive data from server. Both are fundamentally similar. In `REST` web service architecture, `POST` conveys idea of creating new data on server and `PUT` conveys updating of existing data on server. Here we are updating existing `User` using `http PUT`